

Regression analysis, resampling methods and gradient descent for the Runge function

Anne Kristin Furu

Department of Physics, University of Oslo

<https://github.com/Annekfu/FYS-STK4155-Project1>

FYS-STK4155, fall 2026

Abstract

We study ordinary least squares (OLS), Ridge and Lasso regression on Runge’s function $f(x) = 1/(1+25x^2)$ on $[-1, 1]$, fitted with polynomials up to degree 15 on $n = 100$ noisy points. We assess the models with the bootstrap and with k -fold cross-validation, and we replace the closed-form solutions by gradient descent with momentum and the adaptive methods AdaGrad, RMSProp and Adam, both with the full gradient and with minibatches. Our own solvers reproduce the closed-form OLS and Ridge solutions, the analytical gradient matches automatic differentiation to machine precision ($\sim 10^{-15}$), and our proximal Lasso matches scikit-learn to $\sim 10^{-10}$. The bootstrap makes the bias–variance tradeoff explicit: the variance rises steeply with the polynomial degree while the bias stays low. Cross-validation selects the models cleanly, and OLS (degree 12, CV MSE ≈ 0.019) and Ridge (≈ 0.021) are essentially tied, with Lasso a little higher (≈ 0.027). On this smooth, low-dimensional problem the penalties buy little over a well-chosen OLS degree.

1 Introduction

Polynomial regression is the simplest setting in which the central tension of supervised learning appears: a more flexible model fits the training data better but generalises worse. Runge’s function

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1], \quad (1)$$

is a classic hard case for polynomial interpolation, with a sharp central peak that low-degree polynomials cannot follow and high-degree polynomials overfit. It is therefore a good testbed for regression and model selection.

In this project we (i) fit (1) with OLS and Ridge using our own closed-form code; (ii) study the bias–variance tradeoff with the bootstrap and select models with cross-validation; (iii) replace the closed form by gradient descent, with the gradient computed both analytically and by automatic differentiation, and extend it with momentum and the adaptive methods; (iv) solve Lasso, which has no closed form, by proximal gradient descent; and (v) compare OLS, Ridge and Lasso with cross-validation. Throughout we work with scaled data and a fixed random seed for reproducibility.

The report is organised as follows. Section 2 sets up the theory and methods, including the derivation of the bias–variance decomposition. Section 3 describes the code and the validation tests. Section 4 presents and discusses the results part by part, and Section 5 concludes.

2 Theory and methods

2.1 Ordinary least squares

We model the data as $\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}$ with independent noise $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ and a design matrix $\mathbf{X}$ whose columns are powers of x . OLS minimises the mean squared error

$$C(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2, \quad (2)$$

with the closed-form minimiser $\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. Assuming the model is correct, the estimator is unbiased and has variance

$$\mathbb{E}[\hat{\boldsymbol{\theta}}] = \boldsymbol{\theta}, \quad \text{Var}(\hat{\boldsymbol{\theta}}) = \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1}, \quad (3)$$

so the diagonal of (3) gives a confidence interval for each coefficient, with σ^2 estimated from the residuals as $\hat{\sigma}^2 = \|\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}}\|_2^2 / (n - p)$. We solve the normal equations through the singular value decomposition (SVD) rather than an explicit inverse, for numerical stability. The quality of the fit is measured by the MSE and by the R^2 score.

2.2 Ridge regression and singular-value shrinkage

Ridge adds an ℓ_2 penalty, $C(\boldsymbol{\theta}) = \frac{1}{n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|^2 + \lambda \|\boldsymbol{\theta}\|^2$, with the closed form

$$\hat{\boldsymbol{\theta}}_{\text{Ridge}} = (\mathbf{X}^T \mathbf{X} + n\lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}. \quad (4)$$

The factor n comes from the $1/n$ in the data term; scikit-learn omits it, so its `alpha` equals $n\lambda$. Writing $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^T$, the Ridge coefficient in singular direction i is the OLS one multiplied by the filter factor

$$f_i = \frac{s_i^2}{s_i^2 + n\lambda}, \quad (5)$$

so directions with large singular values are kept and small-singular-value directions are shrunk toward zero (Section 3.8 of [3], [5]). These small-singular-value directions are exactly the ill-conditioned ones that make the OLS coefficients unstable.

2.3 Lasso

Lasso replaces the penalty by the ℓ_1 norm, $\lambda \|\boldsymbol{\theta}\|_1$, which has no closed form and is not differentiable at $\theta_j = 0$; its subdifferential there is $[-1, 1]$. We solve it by proximal gradient descent (ISTA): a gradient step on the smooth data term followed by the soft thresholding operator $S_t(z) = \text{sign}(z) \max(|z| - t, 0)$, the proximal operator of the ℓ_1 penalty, which drives coefficients exactly to zero and so yields a sparse model.

2.4 The bias–variance decomposition

Assume the data are generated by $\mathbf{y} = f(\mathbf{x}) + \boldsymbol{\varepsilon}$ with $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \sigma^2)$, and let $\tilde{\mathbf{y}}$ be the prediction of a model fitted on a random training set. Adding and subtracting $\mathbb{E}[\tilde{\mathbf{y}}]$ inside the expected squared error and grouping the terms as $y - \tilde{y} = (f - \mathbb{E}[\tilde{y}]) + \varepsilon + (\mathbb{E}[\tilde{y}] - \tilde{y})$, the three squares survive and the three cross terms vanish, each by one assumption: zero-mean noise, f non-stochastic, and the independence of the test-point noise from the training set. The result is

$$\mathbb{E}[(\mathbf{y} - \tilde{\mathbf{y}})^2] = \underbrace{\frac{1}{n} \sum_i (f_i - \mathbb{E}[\tilde{y}_i])^2}_{\text{Bias}^2} + \underbrace{\frac{1}{n} \sum_i \text{Var}(\tilde{y}_i)}_{\text{Variance}} + \sigma^2. \quad (6)$$

The expectation runs over both the noise and the random training set. The bias measures how far the average model is from the truth, the variance how much the model moves between training sets,

and σ^2 is the irreducible noise. When the true f is unknown we estimate the bias against the noisy test targets, so the measured bias absorbs σ^2 ; the printed identity error = bias + variance then holds per point, with the bias an upper estimate.

2.5 Resampling

The bootstrap estimates (6) by refitting the model on resamples of the training set and reading off the mean, the spread and the error of the predictions on a fixed test set. Cross-validation instead splits the data into k folds, using each fold once as the test set, and averages the test MSE. Any scaling is fitted inside each fold, on the training folds only, so the held-out fold stays unseen.

2.6 Gradient descent and its extensions

Gradient descent updates $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \gamma \nabla C(\boldsymbol{\theta}_k)$ with the analytical gradient $\nabla C_{\text{OLS}} = \frac{2}{n} \mathbf{X}^T (\mathbf{X} \boldsymbol{\theta} - \mathbf{y})$. It converges only for $\gamma < 2/\lambda_{\max}$, where $\lambda_{\max}$ is the largest eigenvalue of the Hessian $\frac{2}{n} \mathbf{X}^T \mathbf{X}$, is fastest at $\gamma^* = 2/(\lambda_{\max} + \lambda_{\min})$, and slows as the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ grows. Momentum, AdaGrad, RMSProp and Adam [4] accelerate or stabilise this. The adaptive methods give each coordinate its own step $\gamma/\sqrt{v_j}$, so γ becomes a length in parameter space and cannot violate the stability bound. Stochastic gradient descent replaces the full gradient by a minibatch gradient inside an epoch loop; the minibatch gradient is unbiased with noise $\propto 1/\sqrt{M}$, so at a constant learning rate the iterate plateaus, and a decaying schedule $\gamma_t = t_0/(t + t_1)$ is needed to converge.

2.7 Automatic differentiation

Automatic differentiation applies the exact chain rule through the code; it is neither symbolic nor a finite difference, so it carries no truncation error, and in reverse mode a full gradient costs only two to three cost evaluations [1, 2]. We use it to check our analytical gradient.

3 Code structure and validation

The code is organised in three folders: **code** (reusable modules and the notebooks), **results** (figures) and **report**. The modules separate the data generation, the three regression methods, the optimisers and the resampling tools, and the five notebooks import them, one per pair of project parts. All randomness uses the seed 2026.

Following the recommendation that a code be checked against known results, we validate every component: our SVD and pseudoinverse OLS agree with each other and with scikit-learn; the analytical gradient matches `jax.grad` to about 10^{-15} for both OLS and Ridge; the Ridge closed form matches scikit-learn once `alpha = n\lambda`; our proximal Lasso matches scikit-learn to about 10^{-10} ; and our own k -fold reproduces the scikit-learn cross-validation.

4 Results and discussion

4.1 OLS (part a)

Figure 1 shows the training and test MSE against the polynomial degree. The training MSE falls monotonically toward the noise floor $\sigma^2 = 0.01$, while the test MSE has a broad minimum around degrees 6–12 and rises again at high degree as the model fits the noise. The coefficient norm grows with the degree, the signature of variance. Scaling does not change the OLS predictions, but it controls the conditioning: Figure 2 shows that the condition number of $\mathbf{X}^T \mathbf{X}$ still reaches $\sim 10^{10}$ at degree 15 even after standardisation, because standardisation removes the difference in column magnitudes but not the strong correlation between the polynomial columns. More data pushes the overfitting to higher degree and lowers the test error.

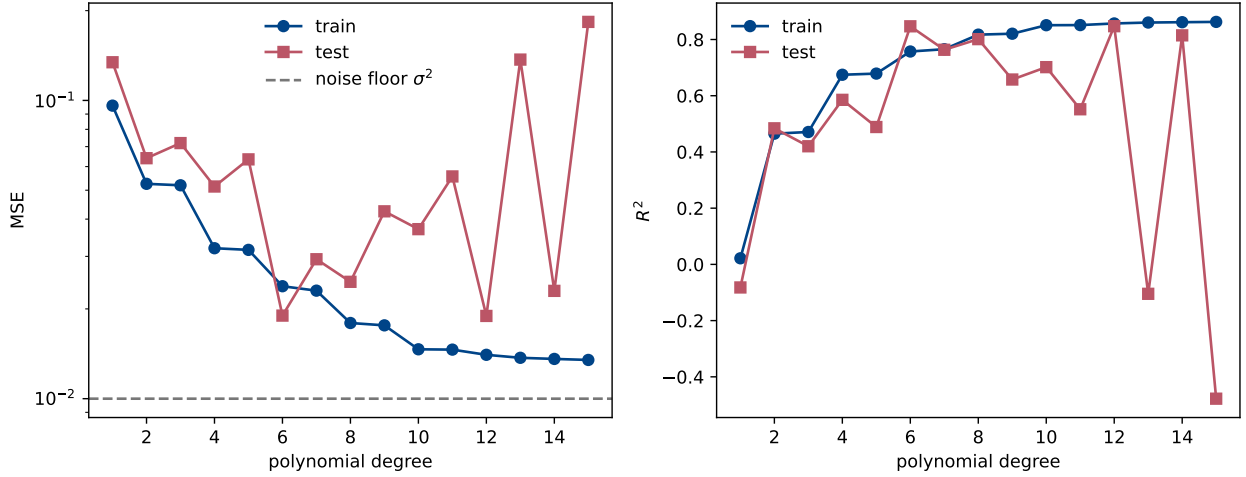


Figure 1: OLS on Runge: training and test MSE (left) and R^2 (right) versus polynomial degree. The dashed line is the noise floor σ^2 .

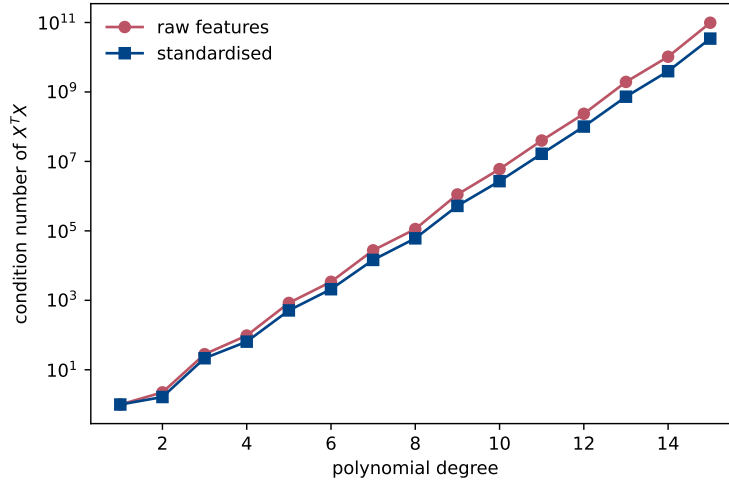


Figure 2: Condition number of $\mathbf{X}^T \mathbf{X}$ versus degree, with and without standardisation. Standardisation helps but does not remove the ill-conditioning at high degree.

4.2 Ridge (part b)

A small penalty lowers the test error below OLS at high degree, while a large penalty underfits. Figure 3 illustrates the mechanism through the filter factors (5): directions with a large singular value are untouched, and small-singular-value directions are shrunk to zero. Ridge does not change the model; it changes which directions the fit trusts, damping the noisy directions that made the OLS coefficients blow up.

4.3 Bias–variance with the bootstrap (part c)

Figure 4 decomposes the bootstrap test error into squared bias and variance. The bias is low and roughly flat, the variance rises steeply with the degree, and their sum gives the U-shaped test error. Repeating for $n = 40, 100, 400$ shows that more data lowers the variance and shifts the overfitting to higher degree, while the bias barely moves: data reduces variance, not bias.

4.4 Cross-validation (part d)

Our own k -fold and the scikit-learn cross-validation agree closely, and both track the bootstrap error and point to the same degree window (Figure 5). At high degree the bootstrap error explodes while

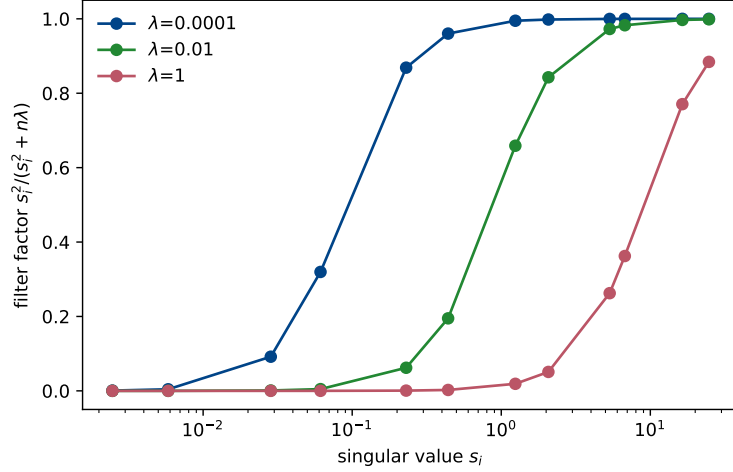


Figure 3: Ridge singular-value shrinkage: the filter factor $s_i^2/(s_i^2 + n\lambda)$ versus the singular value, for three penalties.

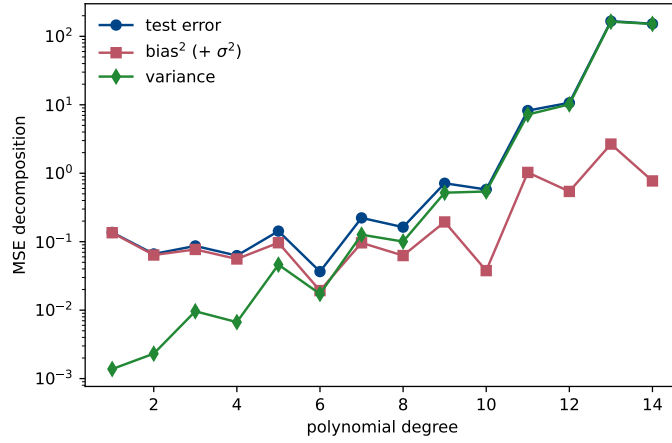


Figure 4: Bias-variance decomposition of the bootstrap test error versus degree ($n = 100$).

the cross-validation stays flat, because a bootstrap resample can be nearly rank-deficient, which destabilises OLS, whereas k -fold keeps distinct points. The selected Ridge penalty is stable between $k = 5$ and $k = 10$.

4.5 Gradient descent and the adaptive methods (parts e, f)

The analytical and automatic gradients agree to machine precision, so either can drive the descent. Figure 6 confirms the stability edge: just below $2/\lambda_{\max}$ the iteration converges, just above it diverges, and γ^* is fastest. On the harder degree-6 problem ($\kappa \approx 2000$), momentum and Adam reach the tolerance in about 1600 and 1300 iterations against 17000 for plain gradient descent (Figure 7); AdaGrad and RMSProp do not reach it at a constant rate. Adam converges over the widest range of learning rates and is the least sensitive to the choice.

4.6 Lasso (part g)

JAX returns 1 for the derivative of $|\theta|$ at zero, a valid subgradient since $1 \in [-1, 1]$; proximal gradient descent is the cleaner tool and sets coefficients exactly to zero. Figure 8 shows the key difference between the penalties: Lasso removes coefficients one by one as λ grows, ending with a sparse model, while Ridge keeps all of them and only shrinks.

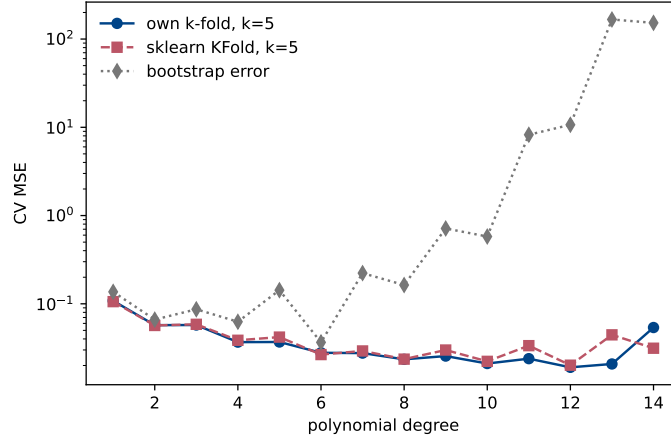


Figure 5: Cross-validated MSE (own k -fold and scikit-learn) versus degree, compared with the bootstrap error.

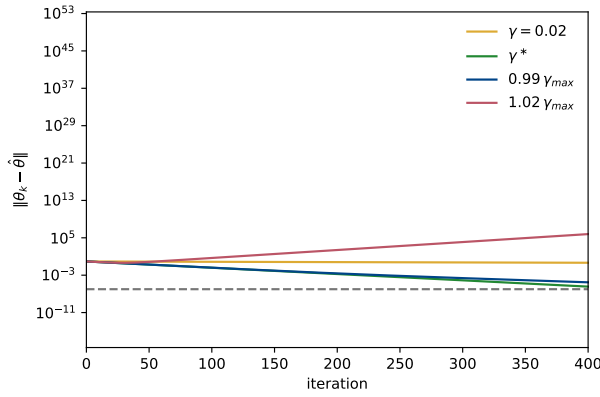


Figure 6: Plain gradient descent for several learning rates (degree 4).

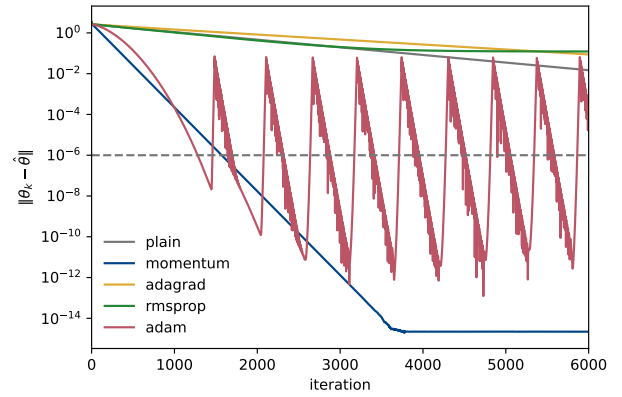


Figure 7: The five optimisers on the degree-6 problem.

4.7 Stochastic gradient descent (part h)

On the well-conditioned degree-2 problem (Figure 9), full-batch gradient descent converges geometrically to machine precision. Constant-rate SGD does not converge; it plateaus at a height that scales with the learning rate, because the iterate is a random walk in a noise cloud. A decaying schedule shrinks that cloud and keeps descending. A single-point step can even diverge in the ill-conditioned case, since it sees that one point’s Hessian, which is why Adam with minibatches is the safe default.

4.8 Final comparison (part i)

Selecting each method by 5-fold cross-validation on Runge gives OLS at degree 12 (CV MSE ≈ 0.019), Ridge at degree 12 with a tiny penalty (≈ 0.021) and Lasso at degree 6 (≈ 0.027). OLS and Ridge are essentially tied and Lasso is slightly higher; all three follow Runge’s peak (Figure 10). Read through (6), OLS controls only the degree, Ridge attacks the variance by shrinking the small-singular-value modes, and Lasso attacks the variance and also cuts the effective complexity by zeroing coefficients. On this smooth, low-dimensional problem with $n \gg p$ the penalties buy little over a cross-validated OLS degree; the sparsity of Lasso would pay off with many irrelevant features.

5 Conclusion and perspectives

We built and validated our own OLS, Ridge and Lasso solvers and a family of gradient-descent optimisers, and applied them to Runge’s function. The bootstrap made the bias–variance tradeoff

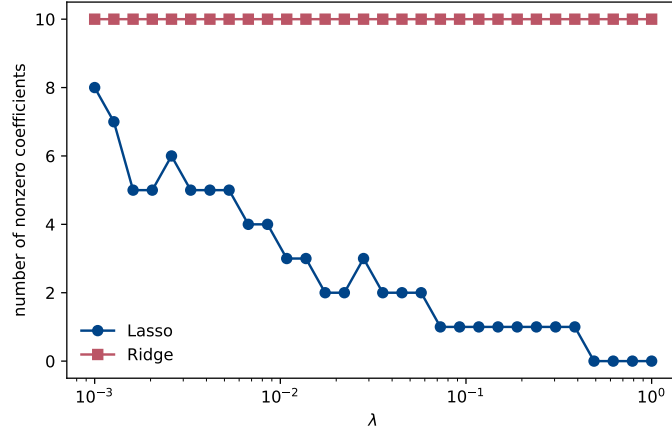


Figure 8: Number of nonzero coefficients versus λ for Lasso and Ridge (degree 10).

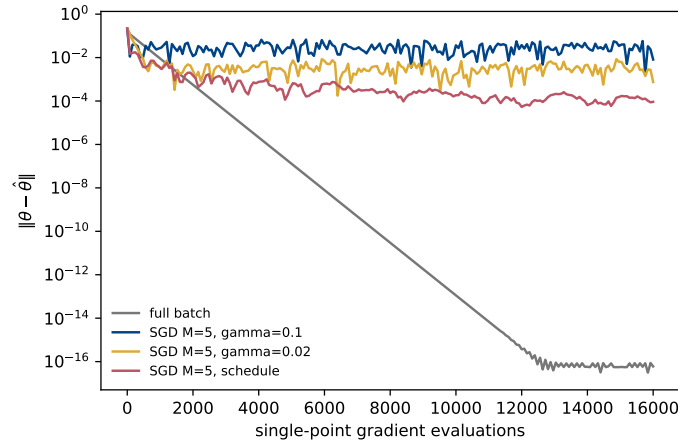


Figure 9: SGD versus full-batch gradient descent (degree 2): constant learning rates plateau, the schedule keeps descending.

explicit, cross-validation selected the models cleanly, and the gradient-descent study reproduced the closed-form solutions and showed how the learning rate, the condition number and the choice of optimiser interact. On this smooth one-dimensional problem OLS, Ridge and Lasso reach almost the same cross-validated error, so regularisation is not decisive here; its value would show on higher-dimensional or noisier data, or with many irrelevant features, which would be a natural next step. A second extension is to replace the linear model by a neural network, keeping the same optimisers and the same cross-validation, where the convexity that made the closed form available is lost and the gradient-descent machinery becomes essential.

Declaration on the use of AI tools

This report and the accompanying code were developed with the assistance of a large language model (Claude, Opus 4.8, from Anthropic). The model was used to scope the project against the weekly exercises, to help structure and write the Python modules and notebooks by porting the author’s own weekly-exercise solutions to the Runge dataset, to generate the figures, and to help draft this report. All prompts and direction came from the author, who has reviewed and verified every result, understands the methods, and is responsible for every line of code and every claim in the report. The libraries used are cited in the bibliography.

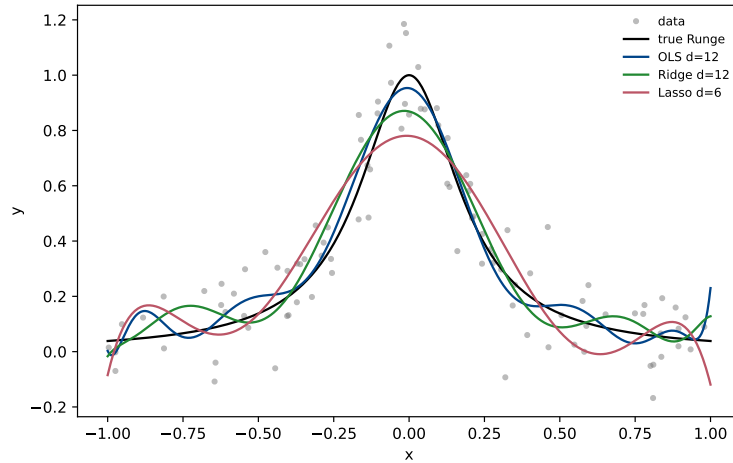


Figure 10: The three cross-validated models refitted on all data, against the true Runge function and the noisy sample.

References

- [1] Atılım Güneş Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic differentiation in machine learning: a survey. *Journal of Machine Learning Research*, 18(153):1–43, 2018.
- [2] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. <http://github.com/google/jax>.
- [3] Morten Hjorth-Jensen. Applied data analysis and machine learning, fys-stk3155/4155 lecture notes, 2026. University of Oslo, <https://github.com/CompPhysics/MachineLearning>.
- [4] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [5] Wessel N. van Wieringen. Lecture notes on ridge regression, 2021. arXiv:1509.09169.